Демо 4. Полный автопилот дайджеста через /loop

Что делаем

Запускаем финальный проект курса: `/loop` крутит `/digest` с интервалом, Stop-хук валидирует обязательные поля выпуска, PostToolUse-хук ведёт лог, CLAUDE.md держит редполитику. Дайджест выходит без человека.

К концу демо у вас есть полностью автоматический блог: Claude ищет новости, пишет статьи, генерирует обложки через Replicate, коммитит в GitHub и деплоит на Vercel, с валидацией качества на каждом шаге.

Подготовка

1. Собраны артефакты прошлых ступеней:
   - Шаблон сайта на Vercel, CLAUDE.md с редполитикой, Memory (ступени 1–4).
   - MCP: Tavily, Replicate, GitHub (ступень 5).
   - Навыки `/digest` и `/cover`, Stop-хук валидации (ступень 6).
   - Команда агентов: `news-scout`, `writer`, `cover-artist`, `page-builder` (ступень 7).
2. Установлен skill `/loop`:

   ```
   # Проверить наличие
   ls ~/.claude/skills | grep loop
   ```

   Если нет — поставить через стандартный механизм установки skills.
3. Репозиторий подключён к Vercel, автодеплой на push в main.
4. Ключи в env: `ANTHROPIC_API_KEY`, `TAVILY_API_KEY`, `REPLICATE_API_TOKEN`, `GITHUB_TOKEN`.

Шаг 1. Проверяем /digest вручную

Запускаем команду из прошлой ступени без `/loop`:

```
/digest
```

Что должно произойти:

1. `news-scout` ищет AI-новости через Tavily, фильтрует по редполитике.
2. `writer` пишет 2–3 статьи.
3. `cover-artist` генерирует обложки через Replicate.
4. `page-builder` собирает MDX-страницы и коммитит.
5. Vercel подхватывает push и деплоит.

Время: 3–5 минут на выпуск. Проверить сайт — должна появиться новая страница.

Шаг 2. Проверяем Stop-хук

Попробуем запустить `/digest` с заведомо плохим результатом: укажем в промпте «не заполняй описание». Stop-хук должен заблокировать завершение сессии с сообщением про пропущенное поле.

Если не блокирует — проверьте `.claude/hooks/validate-article.js`:

```
// Пример структуры хука
const hookData = JSON.parse(fs.readFileSync(0, 'utf-8'));

// Защита от рекурсии
if (hookData.stop_hook_active) process.exit(0);

// Проверка обязательных полей последней статьи
const article = readLatestArticle();
const missing = ['title', 'description', 'cover', 'source']
  .filter(field => !article[field]);

if (missing.length > 0) {
  console.error(`Пропущены поля: ${missing.join(', ')}`);
  process.exit(2);
}

process.exit(0);
```

Exit 2 блокирует завершение, Claude видит stderr и дописывает недостающее.

Шаг 3. Проверяем PostToolUse-хук логов

В `.claude/hooks/pipeline-log.sh`:

```
#!/usr/bin/env bash
# Получаем JSON с данными tool_use
DATA=$(cat)

# Записываем timestamp, имя инструмента, первые 100 символов ввода
echo "$(date -Iseconds) $(echo "$DATA" | jq -r '.tool_name') $(echo
"$DATA" | jq -r '.tool_input' | head -c 100)" >> logs/pipeline.log
```

После прогона `/digest` посмотрите логи:

```
tail -20 logs/pipeline.log
```

Должна быть полная трассировка: `mcp__tavily__search`, `Write app/articles/...`, `mcp__replicate__run`, `Bash(git commit)`, `Bash(git push)`.

Шаг 4. Запускаем /loop

В интерактивном Claude Code:

```
/loop 30m /digest
```

Что делает `/loop`:

- Первый запуск `/digest` начинается сразу.
- После завершения ждёт 30 минут.
- Запускает `/digest` снова.
- Повторяет, пока сессия жива или не прервёте.

В логах и в task list видно, что цикл активен.

Шаг 5. Мониторинг первого часа

Первый час работы — плотно наблюдаем:

1. Откройте `/cost` каждые 10 минут:

   ```
   /cost
   ```

   Одна итерация выпуска должна стоить ~$0.15–0.30 с учётом Sonnet для writer. Если дороже — смотрим, что жрёт: чрезмерное чтение файлов, длинные Tavily-запросы, дорогие Replicate-модели.

2. Откройте `/context`:

   ```
   /context
   ```

   На 70% — пишите в промпт «перед следующим циклом сделай /compact».

3. Проверьте блог:

   ```
   open https://your-digest.vercel.app
   ```

   За час должно появиться 2 выпуска.

4. Проверьте лог пайплайна:

   ```
   tail -100 logs/pipeline.log
   ```

Шаг 6. Обработка ошибок

Типовые сценарии и что делает пайплайн:

Ошибка	Поведение
Tavily вернул пусто	`news-scout` сообщает в task list, следующий цикл через 30 минут.
Replicate дал битую картинку	Stop-хук блокирует — нет `cover`, writer-агент пробует заново.

Git push отклонён	Ошибка в логе, цикл продолжается, следующая итерация повторяет.
Заголовок слишком длинный	Stop-хук блокирует по правилу редполитики, writer переписывает.

Если ошибки повторяются три итерации подряд — `/loop` выдаёт предупреждение, предлагает остановиться.

Шаг 7. Редполитика в CLAUDE.md

Проверьте, что CLAUDE.md задаёт правила, которым следует writer:

```
## Редполитика дайджеста

### Обязательные поля
- `title` — 60 символов, заглавие без точки.
- `description` — одно предложение ≤ 160 символов.
- `cover` — URL обложки с Replicate.
- `source` — ссылка на первоисточник.
- `published_at` — ISO-дата.

### Стиль
- Информационный стиль, без AI-воды.
- Без «от X до Y».
- Предложения ≤ 25 слов.
- Точки в конце каждого буллета.
```

Writer читает CLAUDE.md при старте и проверяет свой вывод по чек-листу.

Шаг 8. Защита от перерасхода

В `.claude/settings.json`:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
```

```
  },
  "permissions": {
    "deny": [
      "Read(.env*)",
      "Read(**/credentials*)",
      "Read(**/*secret*)"
    ]
  },
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "echo \"$CLAUDE_TOOL_INPUT\" | grep -q 'git
push.*main' && echo 'BLOCK: Use /digest branch' && exit 1 || exit 0"
      }]
    }],
    "PostToolUse": [{
      "matcher": ".*",
      "hooks": [{
        "type": "command",
        "command": "bash .claude/hooks/pipeline-log.sh"
      }]
    }],
    "Stop": [{
      "matcher": ".*",
      "hooks": [{
        "type": "command",
        "command": "node .claude/hooks/validate-article.js"
      }]
    }]
  }
}
```

Хук на `git push main` блокирует пуши в main. Пайплайн коммитит в ветку `digest/auto`, потом отдельный шаг мёржит после ручной проверки.

Шаг 9. Останов `/loop`

Варианты:

- `Esc` — прерывает текущую итерацию, ждёт следующую.
- `Esc Esc` — открывает меню, можно остановить `/loop` через skill-команду.
- Закрыть терминал — сессия завершается, `/loop` останавливается.

После остановки Claude распечатывает итоговую статистику: число выпусков, суммарные расходы, ошибки.

Шаг 10. Итоги сессии

Через 3 часа работы:

- 6 выпусков в блоге (30-минутный интервал).
- `logs/pipeline.log` — полная трассировка всех 6 итераций.
- `/cost` — суммарный расход ≤ $2 при правильных моделях.
- Git-история — 6 коммитов в ветке `digest/auto`.

Это и есть финальный harness:

```
/loop 30m /digest
  └── /digest = Agent Teams из 4 агентов
  └── Хуки: Stop (валидация) + PostToolUse (лог) + PreToolUse (защита main)
  └── Контекст: CLAUDE.md (редполитика) + Memory (факты)
  └── Инструменты: Tavily + Replicate + GitHub через MCP
  └── Деплой: Vercel на push
```

Чеклист

- [ ] Ручной `/digest` прошёл полный цикл до деплоя.
- [ ] Stop-хук сработал на заведомо пустом описании статьи.
- [ ] PostToolUse-хук пишет в `logs/pipeline.log` каждый шаг.
- [ ] Skill `/loop` установлен и виден в `/skills`.
- [ ] `/loop 30m /digest` запущен, первая итерация прошла.
- [ ] Через час в блоге ≥ 2 выпусков.
- [ ] `/cost` показывает разумные расходы (≤ $0.30 за выпуск).
- [ ] `/context` на 70% — сделали `/compact`.
- [ ] Deny-список в `settings.json` блокирует чтение `.env` и секретов.
- [ ] PreToolUse-хук блокирует прямой пуш в main.
- [ ] Через 3 часа — 6 выпусков, суммарно ≤ $2.